

Sprint 6 – Software Architecture & Enterprise Java

Study Guide

1. Topic Overview

Topic	What to know
Spring Boot application bootstrap	<code>@SpringBootApplication</code> enables component scanning and auto-configuration. <code>SpringApplication.run(...)</code> starts the application context and embedded server so controllers can be discovered and served.
REST semantics	Use HTTP methods according to their semantics. <code>GET</code> should retrieve data and should not be used to change server state, such as cancelling an order.
OpenAPI and Redocly	A path such as <code>/orders/{id}</code> must define the <code>id</code> path parameter with <code>name</code> , <code>in: path</code> , <code>required: true</code> , and a schema. Specs should also use the correct response schemas and document relevant responses such as authentication failures.
Bean Validation	A DTO received with <code>@RequestBody</code> is not automatically validated just because validation annotations exist on its fields. <code>@Valid</code> is needed to trigger Bean Validation for the request body.
MyBatis result mapping	MyBatis can map selected columns to object fields, including <code>snake_case</code> to <code>camelCase</code> when configured. But it cannot populate a column that was never selected; a missing <code>ticker</code> remains null/unset.
JWT resource-server validation	With a shared HMAC secret, the resource service can verify a JWT signature locally. It does not need to call the identity service for every authenticated request.
Spring Security controller arguments	To inject the authenticated JWT into a controller method, the <code>Jwt</code> parameter should be resolved as the authenticated principal (for example with <code>@AuthenticationPrincipal</code>). Otherwise Spring MVC may try to bind it like a normal request object.
Critical GenAI review	GenAI explanations can sound convincing while being incomplete or subtly wrong about framework internals. Verify suggestions against the framework's actual behavior and the application's configured authentication flow before changing code.
Centralized exception handling	<code>@RestControllerAdvice</code> can handle an exception type regardless of which application layer threw it. One <code>@ExceptionHandler(NoSuchElementException.class)</code> can therefore centralize not-found handling.

Docker networking	Inside a container, `localhost` means that container itself. Containers on the same Docker network should reach Postgres by its container/service name and internal port, e.g. `sprint6-postgres:5432`, rather than the host-published port.
Monolith to microservice	Moving to a networked service introduces concerns such as a request/response contract, persistent state, and authentication/authorization. Existing business arithmetic does not become a new concern merely because transport changes.
Controller-service-repository layering	Layering keeps transport, business logic, and persistence concerns separate. It supports reuse of business logic, makes data-source changes less disruptive to controllers, and allows business logic to be unit tested without a web context.
MyBatis vs. Spring Data JPA	MyBatis makes SQL explicit but usually requires more SQL/boilerplate. JPA can derive common CRUD operations and manages entities through a persistence context with features such as dirty checking.
MyBatis annotations vs. XML	Annotations are convenient for short queries near the mapper method. XML is often clearer for complex joins and dynamic SQL. Snake_case-to-camelCase mapping requires configuration or explicit aliases.
HTTP error status mapping	`MethodArgumentNotValidException` maps naturally to 400 Bad Request; `OrderRejectedException` to 422 Unprocessable Entity when syntax is valid but a business rule rejects it; `NoSuchElementException` to 404 Not Found.
Stateless Spring Security	For a bearer-token REST API, disabling CSRF can be appropriate when cookie-based sessions are not used. `/public` can be permitted while other routes require a valid bearer token. The default JWT converter can be sufficient when custom claims are not needed.
Assembling existing domain logic	Treat an unknown instrument as a not-found condition; a missing holding can represent zero current position when that is the domain rule. Reusing previously isolated validators/updaters shows that business logic need not change just because transport changes.

Multi-stage Docker builds	Do not hardcode <code>localhost</code> for a database running in another container. Use a small JRE-only runtime image rather than shipping Maven/build tools. Copy dependency descriptors before source when using dependency caching so source changes do not invalidate the dependency layer.
End-to-end integration testing	Verify observable behavior across containers: unauthenticated access is rejected, authenticated submission succeeds with a real token, and resulting data is actually persisted in Postgres rather than trusting only the HTTP response.

2. High-Value Memory Aids

Spring Boot — `@SpringBootApplication` discovers/configures; `SpringApplication.run` starts.

REST — `GET` reads; do not use it to mutate server state.

Validation — `@RequestBody` binds; `@Valid` validates.

MyBatis — It maps what SQL returns; it cannot invent an unselected column.

JWT — Shared HMAC secret → local signature verification.

Docker — Inside a container, `localhost` = that container.

Layering — Controller = HTTP, Service = business logic, Repository/Mapper = data.

Errors — 400 = invalid request, 404 = missing resource, 422 = valid request rejected by business rule.

MyBatis vs JPA — MyBatis = explicit SQL; JPA = managed entities and generated/derived persistence behavior.

GenAI — Use AI to investigate, not as unquestioned authority.

3. Question-by-Question Review

Question 1

Why does a Spring Boot REST application need a bootstrap class with `@SpringBootApplication` and `SpringApplication.run(...)` when request handling is in separate controllers?

Study point — Spring Boot Bootstrap: Review the roles of component scanning, auto-configuration, the application context, and the embedded server. Be able to distinguish application startup from request handling.

Exam cue: startup/configuration vs. request handling

Question 2

What is wrong with designing `GET /cancelOrder?id=42` to change an order's state?

Study point — REST Semantics: Know the intended semantics of HTTP methods. Focus on why retrieval operations and state-changing operations should use appropriate verbs, especially when retries and idempotency matter.

Exam cue: HTTP verb semantics and state changes

Question 3

What commonly causes a REDOCLY `path-parameters-defined` violation for a path such as `GET /orders/{id}`?

Study point — OpenAPI Path Parameters: Know the required pieces of an OpenAPI path-parameter definition: its name, location, required status, and schema.

Exam cue: path placeholder must have a complete parameter definition

Question 4

What happens when a Spring controller accepts an `@RequestBody OrderRequestDto` but omits `@Valid`?

Study point — Bean Validation: Separate JSON-to-object binding from Bean Validation. Know what triggers validation annotations on a request DTO and what happens when that trigger is absent.

Exam cue: binding is not the same as validation

Question 5

What happens when a MyBatis query joins an instrument table but does not select the instrument's `ticker` column?

Study point — MyBatis Result Mapping: Understand that MyBatis maps returned columns; naming-convention settings help map column names to fields but do not add columns that the SQL never selected.

Exam cue: mapping can only use columns returned by SQL.

Camel Case Example:

txn_type → txnType

txn_date → txnDate

Question 6

Why can a mission service validate a JWT without calling the identity service for every request?

Study point — JWT Validation: Review how JWT signature verification works when services share the appropriate HMAC secret, and distinguish local token verification from remote authentication calls.

Exam cue: JWTs are designed for verifiable claims

Question 7

Why might a controller parameter of type ``Jwt`` produce a Spring data-binding error even when the bearer token is valid?

Study point — Spring Security Controller Arguments: Know how Spring Security supplies the authenticated principal to controller methods and how an unannotated parameter may instead be treated by Spring MVC as a normal bindable argument.

Exam cue: security principal resolution vs. MVC data binding

Question 8

What risk comes from applying a GenAI-suggested framework fix without checking it against Spring MVC and Spring Security behavior?

Study point — GenAI Review: Treat AI-generated debugging explanations as hypotheses. Verify them against framework behavior, configuration, tests, and the actual authentication flow.

Exam cue: plausible AI output still requires verification

Question 9

Should the same exception type require different `@ExceptionHandler`` methods simply because it can originate in different application layers?

Study point — Centralized Exception Handling: Review how `@RestControllerAdvice`` centralizes exception handling by exception type and helps keep HTTP error translation out of repositories and services.

Exam cue: centralize HTTP translation of shared exception types

Question 10

Why does a containerized service fail when its datasource URL points to ``localhost:5432`` while Postgres is running in another container?

Study point — Docker Networking: Understand what ``localhost`` means inside a container, the difference between host-published and container-internal ports, and service-name-based communication on a Docker network.

Inside a container, **localhost refers to the container itself**, not the host or other containers — the Postgres container must be reached by its container name (ex: sprint6-postgres) and its **internal**, unpublished port (5432), not the host's published port 5433.

Exam cue: container localhost is not another container

Question 11

Which concerns are introduced when a standalone in-process engine becomes a networked microservice?

Study point — Monolith to Microservice: Think about boundaries created by separate processes: contracts, persistence across process lifetimes, and protection from network callers. Distinguish these from business rules that already existed.

Exam cue: network boundaries add distributed-system concerns

Question 12

What benefits come from keeping fee calculation in a service/repository design instead of placing it directly in a controller?

Study point — Controller-Service-Repository Layering: Review separation of concerns, business-logic reuse, testability without a web context, and the ability to change persistence without rewriting HTTP controllers.

Exam cue: separate HTTP, business logic, and persistence

Question 13

Which OpenAPI defects should be caught when reviewing ``GET /orders/{id}``?

Study point — OpenAPI Specification Review: Check path-parameter completeness and whether success and error responses reference schemas appropriate to their meaning.

Exam cue: validate parameters and response schemas

Question 14

What are the important trade-offs between MyBatis and Spring Data JPA?

Study point — MyBatis vs. Spring Data JPA: Compare explicit SQL and CRUD boilerplate in MyBatis with JPA's generated/derived persistence operations, managed entities, persistence context, and dirty checking.

Exam cue: explicit SQL vs. managed persistence

Question 15

When are `@Select` mapper annotations appropriate, and when is XML usually easier to maintain?

Study point — Annotation-Based MyBatis: Know the strengths of annotations for short queries, XML for complex/dynamic SQL, and the role of aliases or underscore-to-camel-case mapping.

Exam cue: simple annotations; complex/dynamic SQL may favor XML

Question 16

How should validation failures, missing resources, and well-formed requests rejected by business rules be represented as HTTP responses?

Study point — HTTP Error Mapping: Be able to distinguish malformed/invalid input, a resource that does not exist, and a syntactically valid request that cannot be processed because of a domain rule.

Exam cue: different failure categories deserve meaningful status codes

Question 17

What does a stateless Spring Security configuration with `/public`, bearer-token authentication, and JWT resource-server support imply?

Study point — Spring Security Configuration: Review CSRF considerations for stateless bearer-token APIs, public versus protected routes, authentication failures, and when the default JWT converter is sufficient.

Exam cue: stateless bearer-token API

Question 18

What practices should be followed when wiring persistence lookups to previously tested `OrderValidator` and `HoldingUpdater` domain logic?

Study point — Assembling Existing Domain Logic: Keep transport and persistence concerns outside reusable domain classes. Decide explicitly how missing instruments and missing holdings map to domain or HTTP behavior.

Exam cue: reuse isolated domain logic

Question 19

What problems should be flagged in a multi-stage Dockerfile for a Spring Boot service?

Study point — Multi-Stage Dockerfile: Review container networking, keeping the runtime image free of unnecessary build tooling, and arranging dependency/source copy steps to take advantage of Docker layer caching.

Exam cue: lean runtime + cache-friendly build + correct networking

Question 20

What should an integration script verify to demonstrate that the containerized system actually works?

Study point — End-to-End Integration Testing: Verify behavior across real boundaries: authentication enforcement, authenticated application flow, and durable database state. Do not treat a successful HTTP status alone as proof of end-to-end correctness.

Exam cue: prove security, flow, and persistence across containers

4. Open API Example

```
@RestController
@RequestMapping("/orders")
@Tag(name = "Orders")
public class OrderController {

    @GetMapping("/{id}")
    @Operation(
        summary = "Get an order by ID",
        description = "Returns an order for the supplied order ID"
    )
    @ApiResponses({
        @ApiResponse(
            responseCode = "200",
            description = "Order found",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(
                    implementation = OrderResponse.class
                )
            )
        ),
        @ApiResponse(
            responseCode = "404",
            description = "Order not found",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(
                    implementation = ErrorResponse.class
                )
            )
        ),
        @ApiResponse(
            responseCode = "401",
            description = "Authentication required"
        )
    })
    public ResponseEntity<OrderResponse> getOrder(

        @Parameter(
            name = "id",
            description = "Unique order ID",
            required = true,
            example = "42"
```

```

    )
    @PathVariable("id") Long id) {

    // retrieve order...
    return ResponseEntity.ok(order);
}
}

```

Redocly

Redocly is a set of tools for working with OpenAPI specifications. In the context of your OpenAPI questions, the important piece is Redocly CLI, an open-source command-line tool that can validate/lint OpenAPI files, enforce API standards, bundle specifications, and generate API documentation.

5. Final Review Checklist

- ☐ Explain why a Spring Boot bootstrap class is needed even when controllers are separate.
- ☐ Identify REST method-semantics problems, especially state-changing GET requests.
- ☐ Spot missing OpenAPI path-parameter definitions and incorrect response schemas.
- ☐ Explain the difference between request binding and Bean Validation with `@Valid`.
- ☐ Predict MyBatis behavior when a selected column is missing or naming conventions do not match.
- ☐ Explain local HMAC JWT verification and authenticated-principal injection.
- ☐ Use `@RestControllerAdvice` for centralized exception handling.
- ☐ Explain Docker container-to-container networking and why `localhost` is usually wrong for another service.
- ☐ Describe the benefits of controller → service → repository layering.
- ☐ Compare explicit MyBatis SQL with JPA persistence-context behavior.
- ☐ Map validation, not-found, and business-rule failures to 400, 404, and 422.
- ☐ Recognize good multi-stage Dockerfile and layer-caching practices.
- ☐ Describe what an end-to-end integration test must actually verify.
- ☐ Critically evaluate GenAI suggestions against framework behavior and existing configuration.

6. One-Minute Summary

Architecture: Keep HTTP, business rules, persistence, security, and infrastructure concerns separated.

Spring: Bootstrap the application context, validate DTOs explicitly, centralize exceptions, and let Spring Security resolve authenticated principals.

Persistence: MyBatis gives explicit SQL and plain mapped objects; JPA manages entities through a persistence context.

Containers: Use service/container names for inter-container communication and keep runtime images lean.

Testing: End-to-end tests should prove authentication, HTTP behavior, and real database persistence—not merely check that a request returned success.